Lab 5 - Listy

lista to elastyczna struktura danych w Pythonie, która pozwala przechowywać i przetwarzać wiele elementów w jednym obiekcie.

Tworzenie „listy” w języku Python

Lista w Pythonie to **wbudowany typ danych**, który **przechowuje uporządkowaną sekwencję elementów**.
Inaczej, to **pojemnik na dane**, w którym ściśle określona jest kolejność danych, ale typ danych może być dowolny

- Może zawierać różne elementy: liczby, napisy, wartości logiczne, a nawet inne listy.
- Jest **mutowalna** – można zmieniać jej zawartość po utworzeniu.
- Możliwe jest utworzenie pustej listy
- W składni, listę definiują nawiasy kwadratowe [ ]

- Przykłady utworzenia listy:

```
owoce = ["jabłko", "banan", "gruszka"]
liczby = [1, 2, 3, 4, 5]
mieszana = [1, "tekst", True, 3.14, liczby]
lista_pusta = []
```

```
# Lista składana - tworzenie listy kwadratów liczb od 0 do 4
x = [i**2 for i in range(5)]
print(x)  # Output: [0, 1, 4, 9, 16]
```

Dostęp (odczyt/zapis) do elementów listy

- Dostęp (odczyt/zapis) do elementów listy jest możliwy pomocą ich **indeksów**.
- Indeks elementu listy określa **pozycja** na liście.
- Indeksowanie zaczyna się od **0**.
  Index pierwszego elementu to **0**.
- Indeksy ujemne np. **-2** oznaczają odliczanie pozycji od końca listy.
  Indeks ostatniego elementu to: **-1**
- Odczyt lub zapis do elementu o wskazanym indeksie jest możliwy jeśli element taki istnieje.
  (Nie można oczytać ani zapisać wartości elementu, który nie istnieje)

```
 8   owoce = ["jabłko", "banan", "gruszka"]
 9   liczby = [1, 2, 3, 4, 5]
10   mieszana = [1, "tekst", True, 3.14, liczby]
11   lista_pusta = []
12
13   Liczby[1] = 20
14   zmienna = liczby[2]
```

Operacje na liście - iteracje

- Odczytanie długości listy, funkcja **len()**

```
16  liczby = [1, 2, 3, 4, 5]
17  dlugosc_listy = len(liczby)
```

- Lista jest **obiektem** „iterowalnym”

```
20  liczby = [1, 2, 7, "owca", 5]
21  for element in liczby:
22      print(element)
23
24  for element in [1, "ala", 3]:
25      print(element)
```

Lista posiada **metody**

- **Metody listy** to wbudowane **funkcje** przypisane do obiektu typu **list**, które pozwalają wykonywać różne operacje na listach – takie jak dodawanie, usuwanie, sortowanie czy wyszukiwanie elementów, itp.
- Dzięki metodom listy nie musimy pisać własnych algorytmów do podstawowych działań, bo Python udostępnia gotowe narzędzia.
- **Wywołanie metody** listy odbywa się poprzez dodanie po nazwie obiektu typu „list” kropki i nazwy metody (wraz z ewentualnymi parametrami wywołania).

```
27   liczby = [5, 8, 3, 6, 12]
28   liczby.sort()
```

- Metoda listy domyślnie operuje na danych listy na której została wywołana.

Metody listy

Metoda	Opis	Przykład
`append(x)`	Dodaje element `x` na końcu listy	`lista.append(5)`
`insert(i, x)`	Wstawia element `x` na pozycję `i`	`lista.insert(1, "kot")`
`extend(iterable)`	Rozszerza listę o elementy innej listy/iterowalnego obiektu	`lista.extend([7, 8])`
`remove(x)`	Usuwa pierwsze wystąpienie elementu `x`	`lista.remove("banan")`
`pop([i])`	Usuwa i zwraca element o indeksie `i` (domyślnie ostatni)	`lista.pop(0)`
`clear()`	Usuwa wszystkie elementy z listy	`lista.clear()`
`index(x)`	Zwraca indeks pierwszego wystąpienia elementu `x`	`lista.index("jabłko")`
`count(x)`	Zlicza, ile razy element `x` występuje w liście	`lista.count(3)`
`sort()`	Sortuje listę w miejscu (rosnąco domyślnie)	`lista.sort()`
`reverse()`	Odwraca kolejność elementów	`lista.reverse()`
`copy()`	Tworzy płytką kopię listy	`nowa = lista.copy()`

Co to jest płytka kopia listy?

Płytka kopia *(ang. **shallow copy**)* to nowa lista, która zawiera odniesienia do tych samych elementów, co lista oryginalna.

- Sama lista (czyli „pojemnik”) jest nowym obiektem.
- Ale elementy w niej nie są kopiowane głęboko – jeśli są obiektami złożonymi (np. inne listy), to wciąż wskazują na te same miejsca w pamięci.

```
30   a = [1, 2, 3]
31   b = a.copy()   # płytka kopia
32   a[0] = 99
33   print(a)  # [99, 2, 3]
34   print(b)  # [1, 2, 3]   -> niezależne, bo elementy są prostymi liczbami
35
36   a = [[1, 2], [3, 4]]
37   b = a.copy()   # płytka kopia
38   a[0][0] = 99
39   print(a)  # [[99, 2], [3, 4]]
40   print(b)  # [[99, 2], [3, 4]]   -> zmiana widoczna także w kopii!
```

Jak w Ptyhon sprawdzić jakie **metody** są dostępne?

- Najprostszy sposób sprawdzenia jakie są metody listy, to użycie wbudowanej funkcji **dir()**, która zwraca listę wszystkich atrybutów i metod obiektu.

```
42  lista = [1, 2, 3]
43  print(dir(lista))
44
45  # ['__add__', '__class__', '__class_getitem__', '__contains__',
46  # '__delattr__', '__delitem__', '__dir__', '__doc__', '__eq__',
47  # '__format__', '__ge__', '__getattribute__', '__getitem__',
48  # '__getstate__', '__gt__', '__hash__', '__iadd__', '__imul__',
49  # '__init__', '__init_subclass__', '__iter__', '__le__', '__len__',
50  # '__lt__', '__mul__', '__ne__', '__new__', '__reduce__',
51  # '__reduce_ex__', '__repr__', '__reversed__', '__rmul__',
52  # '__setattr__', '__setitem__', '__sizeof__', '__str__',
53  # '__subclasshook__', 'append', 'clear', 'copy', 'count',
54  # 'extend', 'index', 'insert', 'pop', 'remove', 'reverse', 'sort']
```

Metody specjalne

Metody zwykłe

Jak sprawdzić opis działania metod?

- Funkcja : help()

```
56  help( lista.append )
57  # Help on built-in function append:
58  #
59  # append(object, /) method of builtins.list instance
60  #    Append object to the end of the list.
```

Wyświetla opis dotyczący składni oraz działania funkcji/metody *(jeśli jest zdefiniowany)*

- Zobacz opis w podręczniku, jeśli jest
- Zapytaj „AI”

Wstawianie elementów do listy

- Wstawianie na koniec: lista.append(x)

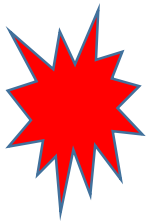
```
62   lista = [3, 1, 4, 1, 5, 9, 2]
63   lista.append(6)
64   print(lista)  # [3, 1, 4, 1, 5, 9, 2, 6]
```

- Wstawianie na wskazanej pozycji: lista.insert(i,x)

```
66   lista = [3, 1, 4, 1, 5, 9, 2]
67   lista.insert(1,100)
68   print(lista)  # [3, 100, 1, 4, 1, 5, 9, 2]
```

- Wstawianie poprzez połączenie list: lista1.extend(lista2)

```
70   lista = [3, 1, 4, 1, 5, 9, 2]
71   lista.extend([50, 51, 52])
72   print(lista)  # [3, 1, 4, 1, 5, 9, 2, 50, 51, 52]
```



- Dodawanie list, operator +, działa tak jak przy dodawaniu napisów

```
48   lista = [1.2,2.3,"ala"]
49   print( lista + [1,"tola",3.7] )
50   # [1.2, 2.3, 'ala', 1, 'tola', 3.7]
```

Usuwanie elementu listy

- Usuwanie pierwszego wystąpienia elementu x : lista.remove(x)

```
74   lista = [3, 1, 4, 1, 5, 9, 2]
75   lista.remove(3)
76   print(lista)  # [1, 4, 1, 5, 9, 2]
```

- Pobieranie i usuwanie elementu z określonej pozycji listy: lista.pop(3)

```
78   lista = [3, 1, 4, 1, 5, 9, 2]
79   wartosc = lista.pop(3)
80   print(wartosc)  # 1
81   Print(lista)  # [3, 1, 4, 5, 9, 2]
```

- Pobieranie i usuwanie ostatniego elementu z listy: lista.pop()

```
83   lista = [3, 1, 4, 1, 5, 9, 2]
84   wartosc = lista.pop()
85   print(wartosc)  # 2
86   print(lista)  # [3, 1, 4, 1, 5, 9]
```

clear(), index(x), count(x)

- Usuwanie wszystkich elementów listy: clear()

```
88   lista = [3, 1, 4, 1, 5, 9, 2]
89   lista.clear()
90   print(lista)  # []
```

- Poszukiwanie indeksu elementu o wartości x: index(x)

```
92   lista = [3, 1, 4, 1, 5, 9, 2]
93   lista.index(1)  # 1 - indeks pierwszego wystąpienia wartości 1
94   print( lista.index(1) )  # 1 - indeks pierwszego wystąpienia wartości 1
```

- Zliczanie liczby wystąpień elementu o wartości x: count(x)

```
96   lista = [3, 1, 4, 1, 5, 9, 2]
97   ilosc_jedynek = lista.count(1)
98   print(ilosc_jedynek)  # 2
99   print( lista.count(7) )  # 0 - brak siódemek w liście
```

Konwersja danych z innego typu do typu „list”

- Python posiada różne typy danych, które przechowują zbiory czy sekwencje danych. Często można je jawnie przekonwertować do typu „list” za pomocą funkcji: **list()**

```
109  napis = "Ala ma kota"
110  print(napis)  # Ala ma kota
111  print(napis[0])  # A
112  print(napis[4])  # m
113  print(napis[-1])  # a
114  lista_znakow = list(napis)
115  print(lista_znakow)
116  # ['A', 'l', 'a', ' ', 'm', 'a', ' ', 'k', 'o', 't', 'a']
```